

Расширение протокола для разработки устройств

Здесь описывается расширение набора команд для упрощения процесса разработки устройств путем введения общего принципа работы с абстрактным набором изменяемых параметров (например, настройки алгоритмов) и абстрактным набором неизменяемых параметров доступных только на чтение (например, телеметрия).

Вводится 4 вида сущностей:

1. набор параметров типа float (IEEE-754, одинарной точности) доступных на чтение и на запись, количество **N1**;
2. набор параметров типа int (знаковое целое, 4 байта) доступных на чтение и на запись, количество **N2**;
3. данные телеметрии типа float (IEEE-754, одинарной точности) доступных только на чтение, количество **N3**;
4. табличные параметры, доступность и формат которых задается **описанием**.

Целочисленные параметры, передаются старшим байтом вперед. Параметры в формате с плавающей точкой **float** и **double** передаются младшим байтом вперед по историческим причинам.

Сценарии работы

Здесь рассматриваются типовые сценарии работы с устройством.

Подключение к устройству

1. Установление соединения канального уровня (соединение TCP или RS-232/422/485)
2. Определение типа устройства с помощью **команды чтения типа и версии устройства 0xFF**. В ответе содержится тип, версия и описание устройства. Если ответа от устройства нет (вышел таймаут на ответ) или устройство возвращает фатальную ошибку (например, `ErrorUnKnowComand` - неизвестная команда), то считается что устройство отсутствует и дальнейшие действия не выполняются, так как эта команда считается обязательной для поддержки устройством.
3. Определение наличия и количества параметров с помощью **команды запроса количества доступных параметров 0xE0**. В ответе содержится 3 целых числа:
 - о кол-во параметров типа float **N1**
 - о кол-во параметров типа int **N2**
 - о кол-во параметров телеметрии **N3**. Если какое-то из количеств **N1**, **N2** или **N3** равно нулю, то

Расширение протокола для разработки устройств

Сценарии работы

Подключение к устройству

Считывание табличного параметра

Список команд

Сообщение об ошибке

0xE0 Запрос количества доступных параметров

0xE1 Записать один параметр типа float

0xE2 Записать один параметр типа int

0xE3 Считать один параметр телеметрии

0xE4 Записать все параметры типа float

0xE5 Записать все параметры типа int

0xE6 Считать все данные телеметрии

0xE7 Получить описание одного параметра типа float

0xE8 Получить описание одного параметра типа int

0xE9 Получить описание одного параметра телеметрии

0xEA Считать один параметр типа float

0xEB Считать один параметр типа int

0xEC Считать все параметры типа float

0xED Считать все параметры типа int

0xEE Считать считать фрагмент описания устройства в формате JSON

0x06 Записать UUID

0x07 Чтение UUID

0x45 Считывание параметров Flash памяти

0x43 Запись страницы Flash памяти

0x44 Чтения страницы Flash памяти

0x46 Получить описание Flash-памяти

0xFA Программный сброс

0xFB Изменение скорости интерфейса

0xFC Изменение адреса Modbus

0xFF Считывание типа и версии устройства

0x50 Запрос кол-ва табличных параметров

0x51 Запрос описания табличного параметра

0x52 Чтение данных табличного параметра

0x53 Запись данных табличного параметра

0x54 Подготовка табличного параметра

0x55 Запрос прогресса подготовки табличного параметра

0x56 Освобождение данных подготовленного табличного параметра

Форматная строка для вывода значений

Формат описания табличных параметров

CRC8

Реализация с использованием таблицы

Реализация без использования таблицы

CRC16

Реализация с использованием таблицы

Реализация без использования таблицы

CRC32

Реализация с использованием таблицы

Реализация с генерацией таблицы

Последовательность обработки команды

Поиск устройства в сети TCP/IPv4

Двоичный формат пакета

Работа байтстаффера

Формат пакета

- соответствующих параметров нет. Если ответа от устройства нет (вышел таймаут на ответ) или устройство возвращает фатальную ошибку, то считается что $N1 = N2 = N3 = 0$.
- Если $N1 > 0$, то нужно получить описание каждого параметра типа float. Это можно сделать с помощью **запроса описания одного параметра типа float 0xE7**. Следует получить описание каждого параметра поочередным вызовом команды 0xE7 для каждого параметра.
 - Если $N2 > 0$, то нужно получить описание каждого параметра типа int. Это можно сделать с помощью **запроса описания одного параметра типа int 0xE8** аналогично предыдущему пункту.
 - Если $N3 > 0$, то нужно получить описание каждого параметра телеметрии. Это можно сделать с помощью **запроса описания одного параметра телеметрии 0xE9** аналогично предыдущему пункту.
 - Определение наличия и количества табличных параметров с помощью **команды запроса количества доступных табличных параметров 0x50**. В ответе содержится целое число - количество табличных параметров NT . Если кол-во 0 или ответа от устройства нет (вышел таймаут на ответ) или устройство возвращает фатальную ошибку, то считается что кол-во табличных параметров 0.
 - Если $NT > 0$, то нужно получить описание каждого табличного параметра с помощью **команды запроса описания табличного параметра 0x51**. Нужно получить описание каждого параметра отдельным вызовом команды 0x51.
 - Определение возможности работы с постоянной памятью (flash) в режиме блочного устройства с помощью **команды чтения параметров Flash памяти 0x45**. В ответе содержится 0 или более пар целых чисел задающих количество страниц **PageCount**, второй - размер страницы в байтах **PageSize**, количество таких пар задает количество таких блочных устройств **NF**. Если ответ пуст (количество пар 0) или ответа от устройства нет (вышел таймаут на ответ) или устройство возвращает фатальную ошибку, то считается что кол-во блочных устройств $NF = 0$.
 - Если $NF > 0$, то можно получить тестовое описание блочного устройства с помощью **команды получения описания Flash-памяти 0x46**. Нужно получить описание каждого блочного устройства отдельным вызовом команды 0x46.
 - Определение возможности работы с постоянной памятью (flash) с разделами с помощью команды получения списка разделов 0x70.**

Считывание табличного параметра

- Подключиться к устройству и считать описание табличных параметров согласно **сценарию подключения к устройству**.
- Если для работы с параметром требуется его подготовка (параметр **NeedParamPrepare** см. **описание табличного параметра**), то перед считыванием нужно подготовить параметр с помощью **команды подготовки табличного параметра 0x54**. После отправки **команды подготовки табличного параметра 0x54** нужно дождаться завершения процесса подготовки параметра с помощью **запроса прогресса подготовки табличного параметра 0x55**, для этого нужно периодически запрашивать **команду 0x55** дождаться момента, когда количество пройденных шагов (прогресс) сравняется с общим количеством шагов. Процедура подготовки табличного параметра используется для тех данных, которые нужно специально подготовить (рассчитать) или "захватить из канала".
- Считывание данных табличного параметра производится с помощью **команды чтения данных табличного параметра 0x52**. С помощью этой команды проводится вычитывание данных табличного параметра небольшими порциями. *Это связано с тем что, данные одного табличного параметра могут занимать несколько мегабайт, и передача такого объема данных одной командой вызовет кучу проблем на практике (канал передачи данных будет занят на долгое время, используемые контрольные суммы сильно теряют свою эффективность при размерах данных превышающих 65 килобайт).* Размер порции данных передаваемых за вызов одной команды 0x52 нужно выбирать с учетом скорости канала передачи, чтобы передача одной команды не занимала канал на секунды.
- Начинаем считывание данных со смещения **offset** = 0 с выбранным размером окна **windowSize**, **wordSize** задает размер слова (размер данных ячейки табличного параметра) в байтах, и он используется в совместно с параметром **stride** для вычитывания данных с пропусками, если это не требуется, то **stride** выбирается равным 1. Размер данных ячейки можно понять из **описания табличного параметра**, поле **Type**. Размер табличного параметра определяется из параметров **Rows** и **Cols** **описания табличного параметра**, задающих количество строк и столбцов таблицы. Для табличных параметров для которых применяется процедура подготовки табличного параметра, описанная в п.2 в ответе на запрос прогресса подготовки табличного параметра, содержится фактическая размерность подготовленной таблицы в полях **rowCount** и **colsCount** ответа.
- запрос данных с помощью **команды чтения данных табличного параметра 0x52**, если ответ получен и контрольная сумма поля данных сошлась, то считаем, что данная порция данных успешно получена. Если ответ не получен или контрольная сумма не сошлась, следует повторно запросить эти данные, если же количество повторных попыток превысит некоторый порог (например, 5), то сдаться с ошибкой.

6. увеличиваем `offset` на `windowSize`, если `offset` достиг размера табличного параметра, то мы получили все данные, если же нет, то переходим к п.5. и запрашиваем следующую порцию данных.
7. Если данные табличного параметра подготавливались согласно п.2, то следует освободить данные табличного параметра с помощью команды освобождения данных табличного параметра `0x56`.

Список команд

Номер	Запрос	Ответ	Описание
1	<code>:AAE0KC<CR></code>	<code>!AAE0xX...XXKC<CR></code>	Запрос количества доступных параметров
2	<code>:AAE1XX...XXKC<CR></code>	<code>!AAE1XX...XXKC<CR></code>	Записать один параметр типа float
3	<code>:AAE2XX...XXKC<CR></code>	<code>!AAE2XX...XXKC<CR></code>	Записать один параметр типа int
4	<code>:AAE3XX...XXKC<CR></code>	<code>!AAE3XX...XXKC<CR></code>	Считать один параметр телеметрии
5	<code>:AAE4XX...XXKC<CR></code>	<code>!AAE4XX...XXKC<CR></code>	Записать все параметры типа float
6	<code>:AAE5XX...XXKC<CR></code>	<code>!AAE5XX...XXKC<CR></code>	Записать все параметры типа int
7	<code>:AAE6KC<CR></code>	<code>!AAE6XX...XXKC<CR></code>	Считать все данные телеметрии
8	<code>:AAE7XX...XXKC<CR></code>	<code>!AAE7XX...XXKC<CR></code>	Получить описание одного параметра типа float
9	<code>:AAE8XX...XXKC<CR></code>	<code>!AAE8XX...XXKC<CR></code>	Получить описание одного параметра типа int
10	<code>:AAE9XX...XXKC<CR></code>	<code>!AAE9XX...XXKC<CR></code>	Получить описание одного параметра телеметрии
11	<code>:AAEAXX...XXKC<CR></code>	<code>!AAEAXX...XXKC<CR></code>	Считать один параметр типа float
12	<code>:AAEBXX...XXKC<CR></code>	<code>!AAEBXX...XXKC<CR></code>	Считать один параметр типа int
13	<code>:AAECKC<CR></code>	<code>!AAECXX...XXKC<CR></code>	Считать все параметры типа float
14	<code>:AAEDKC<CR></code>	<code>!AAEDXX...XXKC<CR></code>	Считать все параметры типа int
15	<code>:AAEEXX...XXKC<CR></code>	<code>!AAEEXX...XXKC<CR></code>	Считать считать фрагмент описания устройства в формате JSON
16	<code>:AA06XX...XXKC<CR></code>	<code>!AA06KC<CR></code>	команда записать UUID
17	<code>:AA07KC<CR></code>	<code>!AA07XX...XXKC<CR></code>	команда чтение UUID
18	<code>:AA45KC<CR></code>	<code>!AA45XX...XXKC<CR></code>	команда чтения параметров Flash памяти
19	<code>:AA43XX...XXKC<CR></code>	<code>!AA43XX...XKC<CR></code>	команда запись страницы Flash памяти
20	<code>:AA44XX...XXKC<CR></code>	<code>!AA44XX...XXKC<CR></code>	команда чтения страницы Flash памяти
21	<code>:AA46XX...XXKC<CR></code>	<code>!AA46XX...XXKC<CR></code>	команда запроса текстового описания Flash памяти
22	<code>:AAFAXXXXXXXXXXKC<CR></code>	<code>!AAFAC<CR></code>	программный сброс
23	<code>:AAFBXX...XXKC<CR></code>	<code>!AAFBXXXXXXXXXXKC<CR></code>	изменение скорости интерфейса
24	<code>:AAffKC<CR></code>	<code>!AAffXX...XXKC<CR></code>	команда чтения типа и версии устройства (доступна с адресом <code>0xFF</code>)

Сообщение об ошибке

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	XX	--	XX	0x0d
ответ	?	XX	XX	Byte	XX	0x0d

Возвращается в ответ на команду которую не удалось корректно обработать. Byte - код ошибки из списка:

<code>_ErrorKC</code>	0x01	КС поля пользователя не верна
<code>_ErrorSize</code>	0x02	ошибочная длина поля пользователя
<code>_ErrorUnKnowType</code>	0x03	неизвестный тип пакета
<code>_ErrorUnKnowComand</code>	0x04	неизвестная команда
<code>_ErrorUnCorrParam</code>	0x05	неверный параметр
<code>_ErrorUnCorrComand</code>	0x06	неверная команда (ошибка при выполнении)

_ErrorDataNotReady	0x07	данные не готовы
_ErrorComandImposWork	0x08	невозможно обработать команду
_ErrorNoSuchParam	0x09	нет такого параметра
_NoError	0xff	Нет ошибки (для внутр. использования)

[Вернуться к краткому описанию команд](#)

0xE0 Запрос количества доступных параметров

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0xE0	Нет	XX	0x0d
ответ	!	XX	0xE0	3*int	XX	0x0d

Запрос количества доступных параметров.

В запросе параметров нет.

В ответе в параметрах сообщается:

1. **N1** количество параметров типа float (int)
2. **N2** количество параметров типа int (int)
3. **N3** количество параметров телеметрии (int)

[Вернуться к краткому описанию команд](#)

0xE1 Записать один параметр типа float

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0xE1	int + float	XX	0x0d
ответ	!	XX	0xE1	int + float	XX	0x0d

Записать один параметр типа float.

В запросе в параметрах сообщается:

1. номер записываемого параметра (int), должен быть меньше **N1** , параметры нумеруются от нуля
2. значение записываемого параметра (float)

В ответе в параметрах сообщается:

1. номер записанного параметра (int)
2. значение записанного параметра (float)

[Вернуться к краткому описанию команд](#)

0xE2 Записать один параметр типа int

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0xE2	int + int	XX	0x0d
ответ	!	XX	0xE2	int + int	XX	0x0d

Записать один параметр типа int.

В запросе в параметрах сообщается:

1. номер записываемого параметра (int), должен быть меньше **N2** , параметры нумеруются от нуля
2. значение записываемого параметра (int)

В ответе в параметрах сообщается:

1. номер записанного параметра (int)

2. значение записанного параметра (int)

[Вернуться к краткому описанию команд](#)

0xE3 Считать один параметр телеметрии

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0xE3	int	XX	0x0d
ответ	!	XX	0xE3	int + float	XX	0x0d

Считать один параметр телеметрии.

В запросе в параметрах сообщается:

1. номер считываемого параметра телеметрии (int), должен быть меньше **N3** , параметры нумеруются от нуля

В ответе в параметрах сообщается:

1. номер считанного параметра (int)
2. значение считанного параметра (float)

[Вернуться к краткому описанию команд](#)

0xE4 Записать все параметры типа float

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0xE4	N1*float	XX	0x0d
ответ	!	XX	0xE4	N1*float	XX	0x0d

Записать все параметры типа float.

В запросе в параметрах сообщается:

1. все параметры типа float в порядке от нулевого до последнего общим количеством **N1** которые надо записать.

В ответе в параметрах сообщается:

1. возвращаются все параметры типа float в порядке от нулевого до последнего общим количеством **N1** которые по факту были записаны.

[Вернуться к краткому описанию команд](#)

0xE5 Записать все параметры типа int

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0xE5	N2*int	XX	0x0d
ответ	!	XX	0xE5	N2*int	XX	0x0d

Записать все параметры типа int.

В запросе в параметрах сообщается:

1. все параметры типа int. в порядке от нулевого до последнего общим количеством **N2** которые надо записать.

В ответе в параметрах сообщается:

1. возвращаются все параметры типа int в порядке от нулевого до последнего общим количеством **N2** которые по факту были записаны.

[Вернуться к краткому описанию команд](#)

0xE6 Считать все данные телеметрии

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0xE6	Нет	XX	0x0d
ответ	!	XX	0xE6	N3*float	XX	0x0d

Считать все параметры телеметрии.

В запросе параметров нет.

В ответе в параметрах сообщается:

- все параметры телеметрии в порядке от нулевого до последнего общим количеством **N3**.

[Вернуться к краткому описанию команд](#)

0xE7 Получить описание одного параметра типа float

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0xE7	int	XX	0x0d
ответ	!	XX	0xE7	int, N*char	XX	0x0d

Получить описание одного параметра типа float.

В запросе в параметрах сообщается:

- номер параметра (int) описание которого запрашивается, должен быть меньше **N1**, параметры нумеруются от нуля

В ответе в параметрах сообщается:

- номер параметра (int)
- массив строк разделенный байтом 0x00 :
 - строка с именем параметра
 - единица измерения параметра
 - форматная строка для вывода на экран значения параметра
 - уникальное имя параметра, допустимые символы:
 - латинские буквы, большие и маленькие
 - цифры
 - символы: ()*+,-./
 - диапазон допустимых значений, нижняя и верхняя граница указанная через символ **;**
 - признак, возможные значения
 - ro** параметр доступный только на чтение
 - wo** параметр доступный только на запись
 - если ничего не указано, то считается что параметр доступен как на чтение, так и на запись.

Формат параметра задается в виде **форматной строки**.

0xE8 Получить описание одного параметра типа int

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0xE8	int	XX	0x0d
ответ	!	XX	0xE8	int, N*char	XX	0x0d

Получить описание одного параметра типа int.

В запросе в параметрах сообщается:

- номер параметра (int) описание которого запрашивается, должен быть меньше **N2**, параметры нумеруются от нуля

В ответе в параметрах сообщается:

1. номер параметра (int)
2. массив строк разделенный байтом 0x00 :
 - строка с именем параметра
 - единица измерения параметра
 - форматная строка для вывода на экран значения параметра
 - уникальное имя параметра, допустимые символы:
 - латинские буквы, большие и маленькие
 - цифры
 - символы: ()*+,-./
 - диапазон допустимых значений, нижняя и верхняя граница указанная через символ ;
 - признак, возможные значения
 - **ro** параметр доступный только на чтение
 - **wo** параметр доступный только на запись
 - если ничего не указано, то считается что параметр доступен как на чтение, так и на запись.

Формат параметра задается в виде **форматной строки**.

0xE9 Получить описание одного параметра телеметрии

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0xE9	int	XX	0x0d
ответ	!	XX	0xE9	int, N*char	XX	0x0d

Получить описание одного параметра телеметрии.

В запросе в параметрах сообщается:

1. номер параметра телеметрии (int) описание которого запрашивается, должен быть меньше **N3** , параметры нумеруются от нуля

В ответе в параметрах сообщается:

1. номер параметра (int)
2. массив строк разделенный байтом 0x00 :
 - строка с именем параметра
 - единица измерения параметра
 - форматная строка для вывода на экран значения параметра
 - уникальное имя параметра, допустимые символы:
 - латинские буквы, большие и маленькие
 - цифры
 - символы: ()*+,-./
 - диапазон допустимых значений, нижняя и верхняя граница указанная через символ ;.

Формат параметра задается в виде **форматной строки**.

[Вернуться к краткому описанию команд](#)

0xEA Считать один параметр типа float

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0xEA	int	XX	0x0d
ответ	!	XX	0xEA	int + float	XX	0x0d

Считать один параметр типа float.

В запросе в параметрах сообщается:

1. номер считываемого параметра (int), должен быть меньше **N1** , параметры нумеруются от нуля

В ответе в параметрах сообщается:

1. номер считанного параметра (int)
2. значение считанного параметра (float)

[Вернуться к краткому описанию команд](#)

0xEB Считать один параметр типа int

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0xEB	int	XX	0x0d
ответ	!	XX	0xEB	int + int	XX	0x0d

Считать один параметр типа int.

В запросе в параметрах сообщается:

1. номер считываемого параметра (int), должен быть меньше **N2** , параметры нумеруются от нуля

В ответе в параметрах сообщается:

1. номер считанного параметра (int)
2. значение считанного параметра (int)

[Вернуться к краткому описанию команд](#)

0xEC Считать все параметры типа float

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0xEC	Нет	XX	0x0d
ответ	!	XX	0xEC	N1*float	XX	0x0d

Считать все параметры типа float.

В запросе параметров нет.

В ответе в параметрах сообщается:

1. все параметры типа float в порядке от нулевого до последнего общим количеством **N1** .

[Вернуться к краткому описанию команд](#)

0xED Считать все параметры типа int

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0xED	Нет	XX	0x0d
ответ	!	XX	0xED	N2*float	XX	0x0d

Считать все параметры типа int.

В запросе параметров нет.

В ответе в параметрах сообщается:

1. все параметры типа int в порядке от нулевого до последнего общим количеством **N2** .

[Вернуться к краткому описанию команд](#)

0xEE Считать считать фрагмент описания устройства в формате JSON

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0xEE	2*int	XX	0x0d

ответ	!	XX	0xEE	3*int, SegSize*byte, int	XX	0x0d
-------	---	----	------	--------------------------	----	------

В запросе в параметрах сообщается:

1. смещение фрагмента в байтах от начала описания Offset (int).
2. максимальный размер фрагмента в байтах для передачи MaxSegSize (int).

В ответе в параметрах сообщается:

1. смещение фрагмента в байтах от начала описания Offset (int).
2. размер фрагмента в байтах SegSize (int) (не более MaxSegSize).
3. размер описания в байтах DesSize (int), значение -1 сообщает о неопределенности размера, в данном случае реальный размер передается в последнем фрагменте.
4. тело фрагмента размером SegSize байт.
5. **контрольная сумма (int)**.

Пример описания в формате JSON [там](#).

[Вернуться к краткому описанию команд](#)

0x06 Записать UUID

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0x06	4 * Integer	XX	0x0d
ответ	!	XX	0x06	нет	XX	0x0d

4 * Integer = 128-ми битный UUID.

Команда используется для завершения калибровки канала, и присвоения каналу уникального идентификатора. Выполняется однократно, после чего становятся не доступными команды 0x01, 0x03, 0x05, 0x06, 0x0F.

[Вернуться к краткому описанию команд](#)

0x07 Чтение UUID

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0x07	нет	XX	0x0d
ответ	!	XX	0x07	4 * Integer	XX	0x0d

[Вернуться к краткому описанию команд](#)

0x45 Считывание параметров Flash памяти

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0x45	нет	XX	0x0d
ответ	!	XX	0x45	(2 * Integer) * N	XX	0x0d

Команда позволяет узнать "геометрию" Flash памяти первый параметр - количество страниц PageCount, второй - размер страницы в байтах PageSize. N - количество доступных флешек.

0x43 Запись страницы Flash памяти

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0x43	(1 + 4 + 4 + PageSize + 1) * Byte	XX	0x0d
ответ	!	XX	0x43	byte + int	XX	0x0d

В запросе: (2 + 4 + PageSize + 1) * Byte - № флешки - 1 байт, № страницы - 4 байта, MagicNumber - 4 байта, страница памяти - PageSize байт, КС - 1 байт;

В ответе: Byte - № флешки, Int - № страницы - 4 байта.

Команда используется для записи страницы Flash памяти. Содержит четыре параметра:

- 1 - номер флешки, в которую необходимо записать данные.
- 2 – номер страницы в которую необходимо записать (диапазон номеров 0...4095), длина номера 4 байта первым передается старший;
- 3 – магическое число (сигнатура), простая защита от случайной записи (число равно 0xEB1C5A3F);
- 4 – содержимое страницы которое необходимо записать (528 байт), нулевой байт страницы передается первым;
- 5 – контрольная сумма полей 1, 2 и 3, рассчитывается как сумма по модулю 256 (один байт) (см. [описание алгоритма](#)).

При получении команду с корректно заполненными полями производится запись во Flash память, проверка записи и в случае успешной записи и корректных параметров возвращается ответ «! XX FD XXXX» иначе «? XX». Страницы 0...3999 отведены под .ldr файл, страницы 4000, 4001, 4002 и 4003 – отведены под сохранение параметров соответственно 1, 2, 3 и 4 каналы), страницы 4004...4095 – резерв.

[Вернуться к краткому описанию команд](#)

0x44 Чтения страницы Flash памяти

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0x44	Byte + Int	XX	0x0d
ответ	!	XX	0x44	(1 + 4 + PageSize + 1) * Byte	XX	0x0d

В запросе: Byte - № флешки, Int - № страницы - 4 байта;

В ответе: (1 + 4 + PageSize + 1) * Byte - № флешки - 1 байт, № страницы - 4 байта, страница памяти - PageSize байт, КС - 1 байт.

Команда используется для чтения Flash памяти и для проверки записи в Flash память.

[Вернуться к краткому описанию команд](#)

0x46 Получить описание Flash-памяти

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0x46	int	XX	0x0d
ответ	!	XX	0x46	int, N*char	XX	0x0d

Получить описание конкретной Flash-памяти

В запросе в параметрах сообщается:

1. номер Flash-карты (int) описание которого запрашивается, должен быть меньше количества Flash-карт из команды 0x45, параметры нумеруются от нуля

В ответе в параметрах сообщается:

1. номер параметра (int)
2. строка описания, оканчивающаяся байтом 0x00

[Вернуться к краткому описанию команд](#)

0xFA Программный сброс

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0xFA	Integer	XX	0x0d
ответ	!	XX	0xFA	Нет	XX	0x0d

Программный сброс контроллера. В запросе в качестве параметра передается "magic number" 0x55AA1234.

[Вернуться к краткому описанию команд](#)

0xFB Изменение скорости интерфейса

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0xFB	2*Integer	XX	0x0d
ответ	!	XX	0xFB	Integer	XX	0x0d

Изменение скорости интерфейса RS485. В запросе поле пользователя состоит из двух полей типа Integer:

1. MagicNumber - "магическое" сочетание чисел 0x12345678
2. скорость интерфейса закодированная следующим образом: 4 байта Integer'a интерпретируются как 4 беззнаковых однобайтных целых числа которые интерпретируются следующим образом:
 1. нулевой (старший) байт - 0x00
 2. первый байт - 0x00
 3. второй байт - дополнение до 0xFF для третьего байта
 4. третий (младший) байт - номер скорости:
 0. 9600 бит/сек;
 1. 19200 бит/сек;
 2. 38400 бит/сек;
 3. 57600 бит/сек;
 4. 115200 бит/сек;
 5. 345600 бит/сек;
 6. 1036800 бит/сек
 7. 230400 бит/сек;
 8. 460800 бит/сек;
 9. 921600 бит/сек;

В ответе сообщается установленная скорость в соответствии с кодировкой для второго Integer'a запроса.

После применения команды новое значение скорости записывается во флеш-память для последующего использования. **Ответ на эту команду передается на "старой" скорости, а следующая команда ожидается на "новой" скорости.**

[Вернуться к краткому описанию команд](#)

0xFC Изменение адреса Modbus

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0xFC	2*Integer	XX	0x0d
ответ	!	XX	0xFC	Integer	XX	0x0d

Изменение адреса устройства на шине Modbus для текущего интерфейса. В запросе поле пользователя состоит из двух полей типа Integer:

1. MagicNumber - "магическое" сочетание чисел 0x23456789
2. адрес закодированный следующим образом: 4 байта Integer'a интерпретируются как 4 беззнаковых однобайтных целых числа которые интерпретируются следующим образом:
 1. нулевой (старший) байт - 0x00
 2. первый байт - 0x00
 3. второй байт - дополнение до 0xFF для третьего байта
 4. третий (младший) байт - адрес устройства от 1 до 254

В ответе сообщается установленный адрес в соответствии с кодировкой для второго Integer'a запроса.

После применения команды новый адрес записывается во флеш-память для последующего использования. **Ответ на эту команду передается со "старого" адреса, а следующая команда ожидается на "новом" адресе.**

[Вернуться к краткому описанию команд](#)

0xFF Считывание типа и версии устройства

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0xFF	нет	XX	0x0d
ответ	!	XX	0xFF	N * Byte	XX	0x0d

Эта команда доступна с по текущему адресу УС и по широковещательному 0xFF.

Команда используется для опознания устройств, строка имеет структуру: 1 и 2 байт – код устройства, 3 и 4 байт версия устройства (это числовые шестнадцатеричные поля), остальные символы текстовое описание устройства, в кодировке Windows-1251. Максимальная длина строки Nmax = 255 символов.

0x50 Запрос кол-ва табличных параметров

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0x50	нет	XX	0x0d
ответ	!	XX	0x50	Integer	XX	0x0d

Запрос количества табличных параметров устройства.

[Вернуться к краткому описанию команд](#)

0x51 Запрос описания табличного параметра

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0x51	Integer	XX	0x0d
ответ	!	XX	0x51	N * Byte	XX	0x0d

Запрос описания табличного параметра с номером N передаваемым в качестве параметра в запросе. В ответе передается описание запрошенного параметра в формате описанном в разделе [Формат описания табличных параметров](#).

Тело описания дополняется контрольной суммой считаеомой по [алгоритму CRC16](#). Контрольная сумма считается от двоичного представления описания параметра.

[Вернуться к краткому описанию команд](#)

0x52 Чтение данных табличного параметра

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0x52	5*Integer	XX	0x0d
ответ	!	XX	0x52	N * Byte	XX	0x0d

Чтение данных табличного параметра. Параметры запроса:

1. номер табличного параметра
2. начальное смещение в байтах `offset`
3. размер запрашиваемых данных в байтах `size`
4. размер поля (слова) в байтах `wordSize`
5. шаг по данным в словах `stride`

Ответ содержит:

1. номер табличного параметра (Integer)
2. начальное смещение в байтах (Integer)
3. кол-во считанных байт тела запрошенного параметра, без КС (т.е. п.4) (Integer)
4. запрошенные данные (N * Byte)
5. контрольная сумма полей 1-4 вычисленная по [алгоритму CRC16](#). (Short)

Начальное смещение и количество запрашиваемых данных задаются в байтах, но должны быть выровнены по размеру поля табличного параметра.

Ответ содержит запрошенное количество данных Nr байт, в случае когда мы доходим до конца табличного параметра возвращаться будет фактически оставшееся количество байт.

[Вернуться к краткому описанию команд](#)

0x53 Запись данных табличного параметра

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0x53	N * Byte	XX	0x0d
ответ	!	XX	0x53	3*Integer	XX	0x0d

Запись данных табличного параметра. Параметры запроса:

1. номер табличного параметра
2. начальное смещение в байтах
3. размер слова (wordSize)
4. шаг (stride)
5. записываемые данные (Nr * Byte)
6. контрольная сумма полей с 1 по 5 включительно вычисленная по [алгоритму CRC16](#). (Short)

Ответ содержит:

1. номер табличного параметра (Integer)
2. начальное смещение в байтах (Integer)
3. кол-во записанных байт запрошенного параметра (Integer)

Начальное смещение и количество запрашиваемых данных задаются в байтах, но должны быть выровнены по размеру поля табличного параметра.

[Вернуться к краткому описанию команд](#)

0x54 Подготовка табличного параметра

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0x54	3*Integer	XX	0x0d
ответ	!	XX	0x54	1*Integer	XX	0x0d

Подготовка табличного параметра. Перед считыванием некоторых табличных параметров необходима операция подготовки. Параметры запроса:

1. номер табличного параметра
2. режим (зарезервированно, передавать 0)
3. номер колонки, которую нужно подготовить (число < 0 означает подготовить все колонки, >= 0 означает подготовить конкретную колонку, число не должно быть равным или превышать [ColsCount](#)).

Ответ содержит:

1. номер табличного параметра (Integer)

[Вернуться к краткому описанию команд](#)

0x55 Запрос прогресса подготовки табличного параметра

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0x55	1*Integer	XX	0x0d
ответ	!	XX	0x55	5*Integer	XX	0x0d

Запрос прогресса подготовки табличного параметра. Параметры запроса:

1. номер табличного параметра

Ответ содержит:

1. номер табличного параметра (Integer)
2. количество пройденных шагов (прогресс) (Integer)
3. общее количество шагов для получения результата (Integer)
4. количество строк в подготовленном табличном параметре `rowCount` (Integer)
5. количество столбцов в подготовленном табличном параметре `colsCount` (Integer)

[Вернуться к краткому описанию команд](#)

команды освобождения данных табличного параметра 0x56

0x56 Освобождение данных подготовленного табличного параметра

	Категория	Адрес	Номер операции	Данные	КС	Возврат каретки
запрос	:	XX	0x56	1*Integer	XX	0x0d
ответ	!	XX	0x56	1*Integer	XX	0x0d

Команда освободить данные подготовленного табличного параметра. Параметры:

1. номер табличного параметра

Ответ содержит:

1. номер освобожденного табличного параметра (Integer)

[Вернуться к краткому описанию команд](#)

Форматная строка для вывода значений

Формат параметра задается в виде **форматной строки для printf**.

К стандартным спецификаторам типа добавлены:

- **%k** - инженерный формат использующий приставки СИ для чисел с плавающей точкой, опции формата задаются в соответствии с форматом **%g**
- **%n** - инженерный формат использующий приставки СИ для чисел с плавающей точкой, опции формата задаются в соответствии с форматом **%f**
- **%t** для вывода данных в форме времени (Дни часы:минуты:секунды.миллисекунды), опций формата нет
- **%T** для вывода данных в формате даты и времени (число-месяц-год часы:минуты:секунды), когда параметр трактуется как отметка времени с начала эпохи Unix, опций формата нет.

Для инженерного формата желательно задавать единицу измерения, язык приставки СИ будет выбираться в соответствии с языком единицы измерения.

Примеры:

- **%.2k** - выводить в инженерном формате число с 2 значащими цифрами

Значение	Форматная строка	Отформатированное значение
100.8	%.2k	100.80
1.04167e+06 Гц	%.2k	1 МГц
1.04167e+06 Гц	%.3k	1.04 МГц
1.04167e+06 Гц	%.4k	1.042 МГц
1.04167e+06 Гц	%.3n	1.042 МГц
1.04167e+06 Гц	%.3g	1.04e+06 Гц
1.04167e+06 Гц	%.3f	1041670.000 Гц
479.833	%t	0:07:59

149500.18	%t	1d 17:31:40
1705238133	%T	14-01-2024 16:15:33

Формат описания табличных параметров

Табличный параметр описывается INI-подобным текстовым файлом.

```
[Table]
Format=1
Access=rw
ParamNum=1
Name=Param1
Desc=Parameter #1
Type=int32
Cols=3
Rows=5
NeedParamPrepare=y
FormatStr=%3d
MinV=0
MaxV=1000

[Column_1]
Title=Column title
MinV=0
MaxV=500
FormatStr=%4d

[Row_1]
Title=Row title
MinV=0
MaxV=200
FormatStr=%5d

[Cell_1_1]
MinV=0
MaxV=300
FormatStr=%6d
```

Секция **Table** должна обязательно присутствовать, содержит описание самой таблицы.

- параметр *Format* - номер версии формата описания, если параметр отсутствует, то подразумевается версия 1, описываемая здесь
- параметр *Access* - режим доступа, если параметр отсутствует, то подразумевается rw (чтение/запись), возможные значения:
 - rw - чтение/запись
 - ro - только чтение
 - wo - только запись
- параметр *ParamNum* - номер описываемого параметра (нумерация параметров начинается с 0)
- параметр *Name* - имя параметра - короткое имя параметра без пробелов, символы из набора ASCII, если параметр не задан, то принимает значение *ParamN*, где N - номер параметра
- параметр *Desc* - текстовое описание параметра, кодировка UTF-8
- параметр *Type* - тип значения ячейки (все ячейки одного типа), обязательный параметр, возможные значения:
 - int8
 - int16
 - int32
 - int64
 - float
 - double
 - fract16
 - fract32
- параметр *Cols* - количество столбцов (обязательный параметр)
- параметр *Rows* - количество строк (обязательный параметр)
- параметр *NeedParamPrepare* - булевский параметр указывающий на необходимость подготовки параметра перед его считыванием (команда 0x54) (по умолчанию подготовка не требуется)
- параметр *FormatStr* - **форматная строка**
- параметр *MinV* - минимально допустимое значение ячейки

- параметр *MaxV* - максимально допустимое значение ячейки

Секция **Column_N** необязательная секция описывает столбец таблицы с номером N, нумерация столбцов от 1 до значения параметра *Cols* из секции **Table**

- параметр *Title* - заголовок столбца, если не указан, то принимается равным номеру столбца
- параметр *MinV* - минимально допустимое значение ячейки, если параметр задан, то его значение перекрывает значение параметра *MinV* в вышестоящей секции **Table**
- параметр *MaxV* - максимально допустимое значение ячейки, если параметр задан, то его значение перекрывает значение параметра *MaxV* в вышестоящей секции **Table**
- параметр *FormatStr* - форматная строка, если параметр задан, то его значение перекрывает значение параметра *FormatStr* в вышестоящей секции **Table**

Секция **Row_N** необязательная секция описывает строку таблицы с номером N, нумерация строк от 1 до значения параметра *Rows* из секции **Table**

- параметр *Title* - заголовок строки, если не указан, то принимается равным номеру строки
- параметр *MinV* - минимально допустимое значение ячейки, если параметр задан, то его значение перекрывает значение параметра *MinV* в вышестоящей секции **Table** и **Column_N**
- параметр *MaxV* - максимально допустимое значение ячейки, если параметр задан, то его значение перекрывает значение параметра *MaxV* в вышестоящей секции **Table** и **Column_N**
- параметр *FormatStr* - форматная строка, если параметр задан, то его значение перекрывает значение параметра *FormatStr* в вышестоящей секции **Table** и **ColumnN**

Секция **Cell_N_M** необязательная секция описывает ячейку таблицы в столбце N и строке M

- параметр *MinV* - минимально допустимое значение ячейки, если параметр задан, то его значение перекрывает значение параметра *MinV* в вышестоящей секции **Table**, **Column_N** и **Row_M**
- параметр *MaxV* - максимально допустимое значение ячейки, если параметр задан, то его значение перекрывает значение параметра *MaxV* в вышестоящей секции **Table**, **Column_N** и **Row_M**
- параметр *FormatStr* - форматная строка, если параметр задан, то его значение перекрывает значение параметра *FormatStr* в вышестоящей секции **Table**, **Column_N** и **Row_M**

CRC8

Используется CRC-8-Dallas/Maxim (из 1-Wire), полином $x^8 + x^5 + x^4 + 1$.

Данная контрольная сумма считается от двоичного представления.

Реализация с использованием таблицы

```
/*
Name   : CRC-8
Poly   : 0x31      x^8 + x^5 + x^4 + 1
Init   : 0xFF
Revert : false
XorOut : 0x00
Check  : 0xF7 ("123456789")
MaxLen : 15 байт (127 бит) - обнаружение
        одинарных, двойных, тройных и всех нечетных ошибок
*/
ushort CRC8 (const uchar *nData, int wLength)
{
    static const uchar wCRCTable[] = {
        0x00, 0x31, 0x62, 0x53, 0xC4, 0xF5, 0xA6, 0x97,
        0xB9, 0x88, 0xDB, 0xEA, 0x7D, 0x4C, 0x1F, 0x2E,
        0x43, 0x72, 0x21, 0x10, 0x87, 0xB6, 0xE5, 0xD4,
        0xFA, 0xCB, 0x98, 0xA9, 0x3E, 0x0F, 0x5C, 0x6D,
        0x86, 0xB7, 0xE4, 0xD5, 0x42, 0x73, 0x20, 0x11,
        0x3F, 0x0E, 0x5D, 0x6C, 0xFB, 0xCA, 0x99, 0xA8,
        0xC5, 0xF4, 0xA7, 0x96, 0x01, 0x30, 0x63, 0x52,
        0x7C, 0x4D, 0x1E, 0x2F, 0xB8, 0x89, 0xDA, 0xEB,
        0x3D, 0x0C, 0x5F, 0x6E, 0xF9, 0xC8, 0x9B, 0xAA,
        0x84, 0xB5, 0xE6, 0xD7, 0x40, 0x71, 0x22, 0x13,
        0x7E, 0x4F, 0x1C, 0x2D, 0xBA, 0x8B, 0xD8, 0xE9,
        0xC7, 0xF6, 0xA5, 0x94, 0x03, 0x32, 0x61, 0x50,
    }
```

```

    0xBB, 0x8A, 0xD9, 0xE8, 0x7F, 0x4E, 0x1D, 0x2C,
    0x02, 0x33, 0x60, 0x51, 0xC6, 0xF7, 0xA4, 0x95,
    0xF8, 0xC9, 0x9A, 0xAB, 0x3C, 0x0D, 0x5E, 0x6F,
    0x41, 0x70, 0x23, 0x12, 0x85, 0xB4, 0xE7, 0xD6,
    0x7A, 0x4B, 0x18, 0x29, 0xBE, 0x8F, 0xDC, 0xED,
    0xC3, 0xF2, 0xA1, 0x90, 0x07, 0x36, 0x65, 0x54,
    0x39, 0x08, 0x5B, 0x6A, 0xFD, 0xCC, 0x9F, 0xAE,
    0x80, 0xB1, 0xE2, 0xD3, 0x44, 0x75, 0x26, 0x17,
    0xFC, 0xCD, 0x9E, 0xAF, 0x38, 0x09, 0x5A, 0x6B,
    0x45, 0x74, 0x27, 0x16, 0x81, 0xB0, 0xE3, 0xD2,
    0xBF, 0x8E, 0xDD, 0xEC, 0x7B, 0x4A, 0x19, 0x28,
    0x06, 0x37, 0x64, 0x55, 0xC2, 0xF3, 0xA0, 0x91,
    0x47, 0x76, 0x25, 0x14, 0x83, 0xB2, 0xE1, 0xD0,
    0xFE, 0xCF, 0x9C, 0xAD, 0x3A, 0x0B, 0x58, 0x69,
    0x04, 0x35, 0x66, 0x57, 0xC0, 0xF1, 0xA2, 0x93,
    0xBD, 0x8C, 0xDF, 0xEE, 0x79, 0x48, 0x1B, 0x2A,
    0xC1, 0xF0, 0xA3, 0x92, 0x05, 0x34, 0x67, 0x56,
    0x78, 0x49, 0x1A, 0x2B, 0xBC, 0x8D, 0xDE, 0xEF,
    0x82, 0xB3, 0xE0, 0xD1, 0x46, 0x77, 0x24, 0x15,
    0x3B, 0x0A, 0x59, 0x68, 0xFF, 0xCE, 0x9D, 0xAC };
uchar crc = 0xFF;

while (wLength--)
    crc = wCRCTable[crc ^ *pData++];

return crc;
}

```

Реализация без использования таблицы

```

uchar crc8(uchar *pBlock, uint len)
{
    uchar crc = 0xFF;
    uint i;

    while (len--) {
        crc ^= *pBlock++;
        for (i = 0; i < 8; i++)
            crc = crc & 0x80 ? (crc << 1) ^ 0x31 : crc << 1;
    }
    return crc;
}

```

CRC16

Используется CRC-16-IBM, он же CRC-16-ANSI (из стандарта Modbus), полином $x^{16} + x^{15} + x^2 + 1$.

Данная контрольная сумма считается от двоичного представления пакета.

Реализация с использованием таблицы

```

/*
Name   : CRC-16
Poly   : 0x8005    x^16 + x^15 + x^2 + 1
Init   : 0xFFFF
Revert : true
XorOut : 0x0000
Check  : 0x4B37 ("123456789")
MaxLen : 4095 байт (32767 бит) - обнаружение
        одинарных, двойных, тройных и всех нечетных ошибок
*/
ushort CRC16 (const uchar *pData, int wLength)
{
    static const ushort wCRCTable[] = {
        0x0000, 0xC0C1, 0x1811, 0x0140, 0x3C01, 0x03C0, 0x0280, 0xC241,
        0xC601, 0x06C0, 0x0780, 0xC741, 0x0500, 0xC5C1, 0xC481, 0x0440,
        0xCC01, 0x0CC0, 0x0D80, 0xCD41, 0x0F00, 0xCFC1, 0xCE81, 0x0E40,
        0x0A00, 0xCAC1, 0xCB81, 0x0B40, 0xC901, 0x09C0, 0x0880, 0xC841,
        0xD801, 0x18C0, 0x1980, 0xD941, 0x1B00, 0xDBC1, 0xDA81, 0x1A40,
        0x1E00, 0xDEC1, 0xDF81, 0x1F40, 0xDD01, 0x1DC0, 0x1C80, 0xDC41,
    }
}

```

```

0x1400, 0xD4C1, 0xD581, 0x1540, 0xD701, 0x17C0, 0x1680, 0xD641,
0xD201, 0x12C0, 0x1380, 0xD341, 0x1100, 0xD1C1, 0xD081, 0x1040,
0xF001, 0x30C0, 0x3180, 0xF141, 0x3300, 0xF3C1, 0xF281, 0x3240,
0x3600, 0xF6C1, 0xF781, 0x3740, 0xF501, 0x35C0, 0x3480, 0xF441,
0x3C00, 0xFCC1, 0xFD81, 0x3D40, 0xFF01, 0x3FC0, 0x3E80, 0xFE41,
0xFA01, 0x3AC0, 0x3B80, 0xFB41, 0x3900, 0xF9C1, 0xF881, 0x3840,
0x2800, 0xE8C1, 0xE981, 0x2940, 0xEB01, 0x2BC0, 0x2A80, 0xEA41,
0xEE01, 0x2EC0, 0x2F80, 0xEF41, 0x2D00, 0xEDC1, 0xEC81, 0x2C40,
0xE401, 0x24C0, 0x2580, 0xE541, 0x2700, 0xE7C1, 0xE681, 0x2640,
0x2200, 0xE2C1, 0xE381, 0x2340, 0xE101, 0x21C0, 0x2080, 0xE041,
0xA001, 0x60C0, 0x6180, 0xA141, 0x6300, 0xA3C1, 0xA281, 0x6240,
0x6600, 0xA6C1, 0xA781, 0x6740, 0xA501, 0x65C0, 0x6480, 0xA441,
0x6C00, 0xACC1, 0xAD81, 0x6D40, 0xAF01, 0x6FC0, 0x6E80, 0xAE41,
0xAA01, 0x6AC0, 0x6B80, 0xAB41, 0x6900, 0xA9C1, 0xA881, 0x6840,
0x7800, 0xB8C1, 0xB981, 0x7940, 0xBB01, 0x7BC0, 0x7A80, 0xBA41,
0xBE01, 0x7EC0, 0x7F80, 0xBF41, 0x7D00, 0xBDC1, 0xBC81, 0x7C40,
0xB401, 0x74C0, 0x7580, 0xB541, 0x7700, 0xB7C1, 0xB681, 0x7640,
0x7200, 0xB2C1, 0xB381, 0x7340, 0xB101, 0x71C0, 0x7080, 0xB041,
0x5000, 0x90C1, 0x9181, 0x5140, 0x9301, 0x53C0, 0x5280, 0x9241,
0x9601, 0x56C0, 0x5780, 0x9741, 0x5500, 0x95C1, 0x9481, 0x5440,
0x9C01, 0x5CC0, 0x5D80, 0x9D41, 0x5F00, 0x9FC1, 0x9E81, 0x5E40,
0x5A00, 0x9AC1, 0x9B81, 0x5B40, 0x9901, 0x99C0, 0x5880, 0x9841,
0x8801, 0x48C0, 0x4980, 0x8941, 0x4B00, 0x8BC1, 0x8A81, 0x4A40,
0x4E00, 0x8EC1, 0x8F81, 0x4F40, 0x8D01, 0x4DC0, 0x4C80, 0x8C41,
0x4400, 0x84C1, 0x8581, 0x4540, 0x8701, 0x47C0, 0x4680, 0x8641,
0x8201, 0x42C0, 0x4380, 0x8341, 0x4100, 0x81C1, 0x8081, 0x4040 };

ushort wCRCWord = 0xFFFF;
uchar nTemp;

while (wLength--) {
    nTemp = *pData++ ^ wCRCWord;
    wCRCWord >>= 8;
    wCRCWord ^= wCRCTable[nTemp];
}
return wCRCWord;
}

```

Используется CRC-32 (

Реализация без использования таблицы

```

uint CRC16_2(const char *buf, int len)
{
    uint crc = 0xFFFF;

    for (int pos = 0; pos < len; pos++) {
        crc ^= (uint)buf[pos];           // XOR byte into least sig. byte of crc

        for (int i = 8; i != 0; i--) {    // Loop over each bit
            if ((crc & 0x0001) != 0) {    // If the LSB is set
                crc >>= 1;                // Shift right and XOR 0xA001
                crc ^= 0xA001;
            } else                        // Else LSB is not set
                crc >>= 1;                // Just shift right
        }
    }
    // Note, this number has low and high bytes swapped, so use it accordingly (or swap bytes)
    return crc;
}

```

CRC32

Используется CRC-32 (Ethernet, SATA, ZModem), полином $x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$.

Данная контрольная сумма считается от двоичного представления пакета.

Реализация с использованием таблицы

```

/*
  Name   : CRC-32
  Poly   : 0x04C11DB7       $x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11}$ 
                         $+ x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$ 

  Init   : 0xFFFFFFFF
  Revert : true
  XorOut : 0xFFFFFFFF
  Check  : 0xCBF43926 ("123456789")
  MaxLen : 268 435 455 байт (2 147 483 647 бит) - обнаружение
            одинарных, двойных, пакетных и всех нечетных ошибок
*/
ushort CRC32 (const uchar *nData, int wLength)
{
    static const uint wCRCTable[] = {
        0x00000000UL, 0x77073096UL, 0xEE0E612CUL, 0x990951BAUL,
        0x076DC419UL, 0x706AF48FUL, 0xE963A535UL, 0x9E6495A3UL,
        0x0EDB8832UL, 0x79DCB8A4UL, 0xE0D5E91EUL, 0x97D2D988UL,
        0x09B64C2BUL, 0x7EB17CBDUL, 0xE7B82D07UL, 0x90BF1D91UL,
        0x1DB71064UL, 0x6AB020F2UL, 0xF3B97148UL, 0x84BE41DEUL,
        0x1ADAD47DUL, 0x6DDDE4EBUL, 0xF4D4B551UL, 0x83D385C7UL,
        0x136C9856UL, 0x646BA8C0UL, 0xFD62F97AUL, 0x8A65C9ECUL,
        0x14015C4FUL, 0x63066CD9UL, 0xFA0F3D63UL, 0x8D088DF5UL,
        0x3B6E20C8UL, 0x4C69105EUL, 0xD56041E4UL, 0xA2677172UL,
        0x3C03E4D1UL, 0x4B04D447UL, 0xD20D85F9UL, 0xA50AB56BUL,
        0x35B5A8FAUL, 0x42B2986CUL, 0xDBBBC9D6UL, 0xACBCF940UL,
        0x32D86CE3UL, 0x45DF5C75UL, 0xDCD60DCFUL, 0xABD13D59UL,
        0x26D930ACUL, 0x51DE003AUL, 0xC8D75180UL, 0xBFDD06116UL,
        0x21B4F4B5UL, 0x56B3C423UL, 0xCFBA9599UL, 0xB8BDA50FUL,
        0x2802B89EUL, 0x5F058808UL, 0xC60CD9B2UL, 0xB10BE924UL,
        0x2F6F7C87UL, 0x58684C11UL, 0xC1611DABUL, 0xB6662D3DUL,
        0x76DC4190UL, 0x01DB7106UL, 0x98D220BCUL, 0xEFDD5102AUL,
        0x71B18589UL, 0x06B6B51FUL, 0x9FBBFE4A5UL, 0xE8B8D433UL,
        0x7807C9A2UL, 0x0F00F934UL, 0x9609A88EUL, 0xE10E9818UL,
        0x7F6A0DBBUL, 0x086D3D2DUL, 0x91646C97UL, 0xE6635C01UL,
        0x6B6B51F4UL, 0x1C6C6162UL, 0x856530D8UL, 0xF262004EUL,
        0x6C0695EDUL, 0x1B01A57BUL, 0x8208F4C1UL, 0xF50FC457UL,
        0x65B0D9C6UL, 0x12B7E950UL, 0x8BBEB8EAUL, 0xFCB9887CUL,
        0x62DD1DDFUL, 0x15DA2D49UL, 0x8CD37CF3UL, 0xFBD44C65UL,
        0x4DB26158UL, 0x3AB551CEUL, 0xA3BC0074UL, 0x74BB30E2UL,
        0x4ADFA541UL, 0x3DD895D7UL, 0xA4D1C46DUL, 0x73D6F4FBUL,
        0x4369E96AUL, 0x346ED9FCUL, 0xAD678846UL, 0xDA60B8D0UL,
        0x44042D73UL, 0x33031DE5UL, 0xAA0A4C5FUL, 0xDD0D7CC9UL,
        0x5005713CUL, 0x270241AAUL, 0xBE0B1010UL, 0xC90C2086UL,
        0x5768B525UL, 0x206F85B3UL, 0xB966D409UL, 0xCE61E49FUL,
        0x5EDEF90EUL, 0x29D9C998UL, 0xB0D09822UL, 0xC7D7A8B4UL,
        0x59B33D17UL, 0x2EB40D81UL, 0xB7BD5C3BUL, 0xC0BA6CADUL,
        0xEDB88320UL, 0x9ABFB3B6UL, 0x03B6E20CUL, 0x74B1D29AUL,
        0xEAD54739UL, 0x9DD277AFUL, 0x04DB2615UL, 0x73DC1683UL,
        0xE3630B12UL, 0x94643B84UL, 0x0D6D6A3EUL, 0x7A6A5AA8UL,
        0xE40ECF0BUL, 0x9309FF9DUL, 0x0A00AE27UL, 0x7D079EB1UL,
        0xF00F9344UL, 0x8708A3D2UL, 0x1E01F268UL, 0x6906C2FEUL,
        0xF762575DUL, 0x806567CBUL, 0x196C3671UL, 0x6E6B06E7UL,
        0xFED41B76UL, 0x89D32BE0UL, 0x10DA7A5AUL, 0x67DD4ACCUL,
        0xF9B9DF6FUL, 0x8EBEEFF9UL, 0x17B7BE43UL, 0x60B0BED5UL,
        0xD6D6A3E8UL, 0xA1D1937EUL, 0x38D8C2C4UL, 0x4FDDF252UL,
        0xD1BB67F1UL, 0xA6BC5767UL, 0x3FB506DDUL, 0x48B2364BUL,
        0xD80D2BDAUL, 0xAF0A1B4CUL, 0x36034AF6UL, 0x41047A60UL,
        0xDF60EFC3UL, 0xA867DF55UL, 0x316E8EEFUL, 0x4669BE79UL,
        0xCB61B38CUL, 0xBC66831AUL, 0x256FD2A0UL, 0x5268E236UL,
        0xCC0C7795UL, 0xBB0B4703UL, 0x220216B9UL, 0x5505262FUL,
        0xC5BA3BBEUL, 0xB2BD0B28UL, 0x2BB45A92UL, 0x5CB36A04UL,
        0xC2D7FFA7UL, 0xB5D0CF31UL, 0x2CD99E8BUL, 0x5BDEAE1DUL,
        0x9B64C2B0UL, 0xEC63F226UL, 0x756AA39CUL, 0x026D930AUL,
        0x9C0906A9UL, 0xEB0E363FUL, 0x72076785UL, 0x05005713UL,
        0x95BF4A82UL, 0xE2B87A14UL, 0x7BB12BAEUL, 0x0CB61B38UL,
        0x92D28E9BUL, 0xE5D5BE0DUL, 0x7CDCEFB7UL, 0x0BDBDF21UL,
        0x86D3D2D4UL, 0xF1D4E242UL, 0x68DDB3F8UL, 0x1FDA836EUL,
        0x81BE16CDUL, 0xF6B9265BUL, 0x6FB077E1UL, 0x18B74777UL,
        0x88085AE6UL, 0xFF0F6A70UL, 0x66063BCAUL, 0x11010B5CUL,
        0x8F659EFFUL, 0xF862AE69UL, 0x616BFFD3UL, 0x166CCF45UL,
        0xA00AE278UL, 0xD70DD2EEUL, 0x4E048354UL, 0x3903B3C2UL,
        0xA7672661UL, 0xD06016F7UL, 0x4969474DUL, 0x3E6E77DBUL,
        0xAED16A4AUL, 0xD9D65ADCUL, 0x40DF0B66UL, 0x37D83BF0UL,
        0xA9BCAE53UL, 0xDEBB9EC5UL, 0x47B2CF7FUL, 0x30B5FFE9UL,
        0xBDBDF21CUL, 0xCABAC28AUL, 0x53B39330UL, 0x24B4A3A6UL,
        0xBAD03605UL, 0xCDD70693UL, 0x54DE5729UL, 0x23D967BFUL,
    
```



```

        0xB3667A2EUL, 0xC4614AB8UL, 0x5D681B02UL, 0x2A6F2B94UL,
        0xB40BBE37UL, 0xC30C8EA1UL, 0x5A05DF1BUL, 0x2D02EF8DUL };
uint crc = 0xFFFFFFFF;
uchar nTemp;

while (wLength--)
    crc = (crc >> 8) ^ wCRCTable[(crc ^ *pData++) & 0xFF];

return crc^0xFFFFFFFF;
}

```

Реализация с генерацией таблицы

```

uint_least32_t Crc32(uchar *buf, size_t len)
{
    uint_least32_t crc_table[256];
    uint_least32_t crc;
    int i, j;

    for (i = 0; i < 256; i++) {
        crc = i;
        for (j = 0; j < 8; j++)
            crc = crc & 1 ? (crc >> 1) ^ 0xEDB88320UL : crc >> 1;
        crc_table[i] = crc;
    };
    crc = 0xFFFFFFFFUL;
    while (len--)
        crc = crc_table[(crc ^ *buf++) & 0xFF] ^ (crc >> 8);

    return crc ^ 0xFFFFFFFFUL;
}

```

Последовательность обработки команды

При получении пакета сверяется контрольная сумма, если она не сошлась, то пакет отбрасывается, и в ответ ничего не шлем. Дальше, если адрес назначения не совпадает с нашим адресом, то просто молчим, единственное исключение команда запроса типа и версии 0xFF, которая доступна как по индивидуальному адресу, так и по широковещательному адресу 0xFF. Дальше если номер команды нам неизвестен, то отвечаем BadCommand.

Продолжение следует...

Поиск устройства в сети TCP/IPv4

Возможность поиска совместимых устройств в сети TCP/IP (IP протокол версии 4) реализована с помощью отправки запроса в на широковещательный адрес 239.1.2.1, порт 5001. Получившие этот запрос устройства передают в ответ структуру с основными своими параметрами.

Запрос имеет фиксированное содержание размером 4 байта:

0	1	2	3
0x46	0x49	0x4E	0x45

Ответ имеет следующую структуру:

```

const int kIpSearchDescriptionMaxLen = 100;
const int kIpSearchSerialMaxLen     = 40;
const uint32 kIpSearchMagic          = 0x34125643;
struct answer {
    uint32  magic;
    uint16  dataVersion, serverPort;
    int32   modbusAddress;
    uint16  type, version;
    char    description[kIpSearchDescriptionMaxLen];
    char    serialNumber[kIpSearchSerialMaxLen];
};

```

Содержимое полей структуры ответа **answer**:

- **magic** должно содержать значение **0x34125643**
- **dataVersion** - версия структуры ответа, должна быть **1**
- **serverPort** - порт на котором принимаются входящие подключения по протоколу TCPv4
- **modbusAddress** - адрес в сети Modbus на который отвечает устройство
- **type** - тип устройства, соответствует типу из ответа на команду **запроса типа и версии устройства**
- **version** - версия устройства, соответствует версии из ответа на команду **запроса типа и версии устройства**
- **description** - описание устройства, соответствует описанию из ответа на команду **запроса типа и версии устройства**
- **serialNumber** - серийный номер устройства, сейчас в качестве серийного номера передается UUID (см. команды **записать UUID** и **считать UUID**)

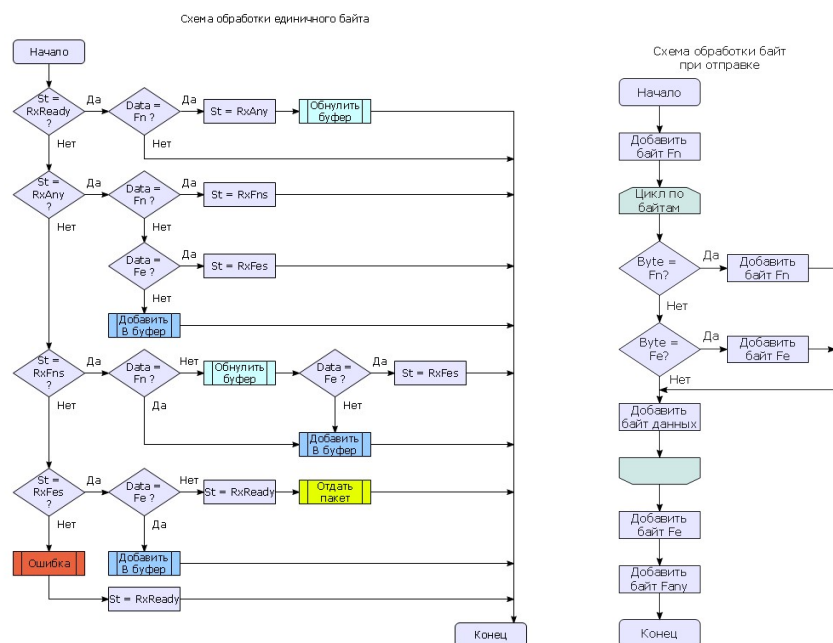
Двоичный формат пакета

Передача ведется в бинарном режиме. Для выделения пакета зарезервированны символы начала пакета **0x55** и окончания пакета **0xAA**, если эти байты встречаются в теле пакета, то они удваиваются. После символа конца пакета нужно передать любой байт отличный от символа конца пакета, обычно это байт **0xFF**.

- **0x55** - StartByte - Символ/флаг начала пакета
- **0xAA** - EndByte - Символ/флаг окончания пакета
- **0xFF** - AnyByte - Символ/флаг необходимый для правильной обработки символа окончания пакета

Работа байтстаффера

Блок-схема работы байтстаффера для двоичного протокола (из проекта FreGraf):



Формат пакета

0	1	2	3	4	5	...	N-1	N	N+1	N+2
StartByte	Address	PacketNumber	PacketType	HeaderCRC	Command	CommandBody	CommandCRC	EndByte	AnyByte	

- StartByte - флаг начала пакета **0x55**
- Address - адрес устройства от 0 до 254, широковещательный адрес 255
- PacketNumber - номер пакета от 0 до 255, в каждом новом запросе номер пакета увеличивается на 1, номер пакета ответа или ошибки соответствует номеру в пакете запроса

- PacketType - тип пакета:
 - 0x00 - Request - запрос
 - 0x01 - Answer - ответ
 - 0x02 - Error - ошибка
- HeaderCRC - контрольная сумма заголовка **CRC8**, считается по полям Address, PacketNumber и PacketType
- Command - номер команды от 0 до 255
- CommandBody - тело команды
- CommandCRC - контрольная сумма тела пакета **CRC16**, считается по полям Command и CommandBody
- EndByte - флаг конца пакета **0xAA**
- AnyByte - **0xFF**

Вложения (2)

Последнее изменение: 25 июл. 2024 г., 10:48:41